

ORCHESTRA-OS: Implementation and Experimental Validation

S.W. ZAW

Final Systems Software Implementation Report

14 September 2026

Abstract

This report documents the implementation and experimental validation of a Linux scheduling prototype derived from the ORCHESTRA-OS architecture proposed by Ganesan and Vaithianathan. The engineering deliverable comprises an opt-in scheduler, userspace bridge, versioned BPF maps, task directives, and runtime telemetry. Archived kernel experiments demonstrate attachment, task ownership, five observable actions, and clean unload. In 20 counterbalanced foreground/background pairs, mean foreground completion time decreased from 7.988 to 2.037 seconds: a 74.24% mean paired reduction, accompanied by 97.69% lower background CPU service. Pure-CPU fixed-work comparisons were slower than the baseline, with ratios of 1.58–2.05; mixed-workload ratios were 0.86–0.94. Three 600-second stress phases completed with ownership evidence and zero reported errors. The 14 September validation recorded 30/30 named unit tests, successful integration and security suites, and 15 completed CFS baseline rows. Its ORCHESTRA branch produced no valid timing rows because attachment prerequisites were unmet. The evidence supports a partially validated research prototype and a measured foreground-service trade-off; it does not establish general scheduler superiority.

Keywords: Linux scheduling; sched_ext; eBPF; foreground protection; experimental evaluation.

1 Introduction

Ganesan and Vaithianathan’s 6 July 2026 submission draft establishes the architecture and pre-kernel simulation basis for this work [1]. Their contributions include predictive signal coordination, the corrected four-factor coordination metric, adaptive responses, and controller design principles. The present report documents the software implementation and its measured behavior. Architectural authorship and simulation findings belong to the cited authors.

The implementation questions are whether admitted tasks enter the scheduler, whether RUN, SLEEP, MIGRATE, THROTTLE, and YIELD have observable effects, and what workload costs and benefits follow. Existing reports [2, 3, 5, 6, 8] supply the evidence. No new runtime campaign was executed during preparation of this report; simulation values are not used as hardware acceptance thresholds.

2 Experimental platform and evidence

The experiments used a ThinkPad T14 Gen 1 with an Intel Core i5-10310U, four physical cores, eight logical CPUs, approximately 31 GiB RAM, and one NUMA node. The Linux kernel was 7.0.12+kali-amd64. The recorded source revision for the principal archived comparison was 94664aeb001d; the fresh 14 September baseline used revision e436eb517ec9. The working tree contained local changes, so build manifests and archived artifact hashes identify the tested binaries more precisely than a commit alone.

The baseline is labeled CFS in the experiment records. ORCHESTRA workload interpretation required evidence that the target task entered the sched_ext path. Commands, return codes, standard output, standard error, and telemetry were retained in the campaign archives. The September runs were executed on a host with an active graphical desktop session and ordinary background services; the machine was not an isolated benchmark node. This environmental condition is disclosed as a confounder, and the September timing measurements are interpreted separately from the earlier fixed-work protocol.

3 Scheduler workflow

Figure 1 summarizes the implemented single-host control path described in the existing architecture documentation and runtime reports [3, 7]. It distinguishes publication and admission from task execution and observation.

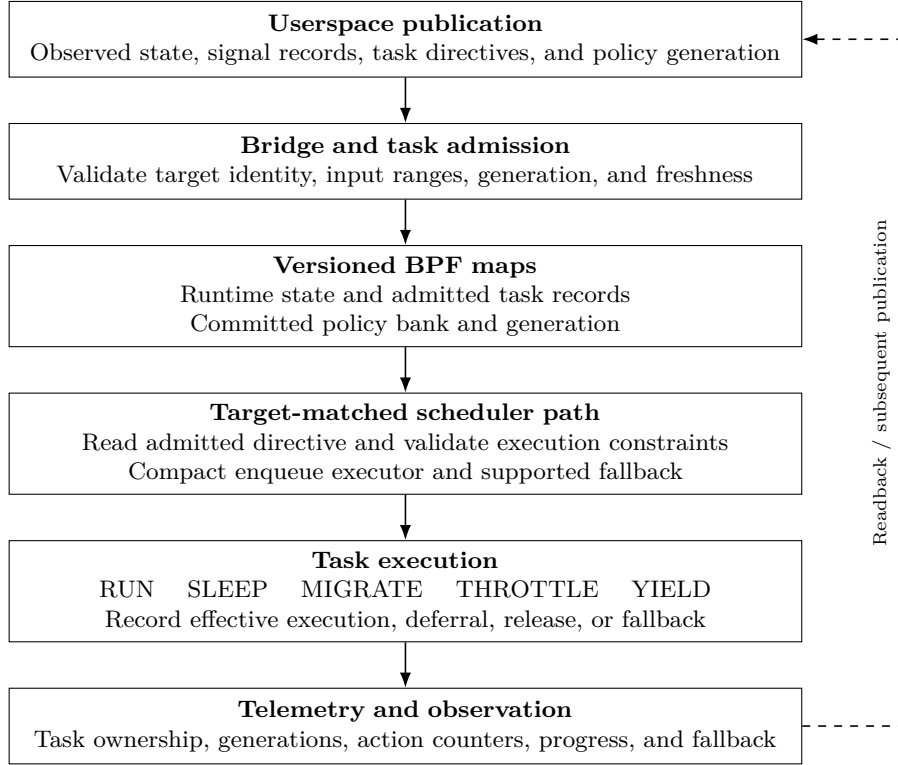


Figure 1: Runtime workflow for the archived target-matched prototype. The dashed return denotes telemetry readback and subsequent publication. Policy-bank storage and prediction fields do not, by themselves, prove online learning, forecast accuracy, or controller convergence.

3.1 Admission and execution

Loading the scheduler and admitting a task are separate operations. The bridge validates the target identity and publishes the task record. Runtime probes then correlate task-level counters with forward progress. Action selection is checked against backend capabilities; unsupported selections can fall back to RUN. Deferred SLEEP and THROTTLE behavior is evaluated through both deferral and subsequent release.

3.2 Telemetry as experimental evidence

The observations distinguish requested, accepted, dispatched, running, effective, and fallback outcomes. These counters describe different events and need not have identical totals. For example, a request alone does not establish CPU movement. The migration probes therefore include the requested target, dispatched target, and observed CPU. Generation-aware throttle records additionally preserve charged runtime, eligibility, and period state [7, 3].

4 Workload performance

4.1 Foreground protection

The held-out photo/archive experiment paired an ImageMagick foreground workload with a bzip2 background workload. Twenty pairs were counterbalanced: ten ran ORCHESTRA first and ten ran the Linux baseline first. All 40 workload runs exited successfully and passed output verification; all 20 ORCHESTRA phases passed ownership and effective-deferral checks [2].

Table 1: Foreground protection experiment: 20 paired trials.

Measurement	Linux baseline	ORCHESTRA
Foreground mean completion time (ms)	7,987.80	2,036.60
Foreground sample SD (ms)	1,202.55	297.04
Mean background CPU ticks	4,872.85	112.80

For pair i , foreground improvement is $r_i = 100(T_{L,i} - T_{O,i})/T_{L,i}$. Its mean was 74.24%, with sample SD 3.25 percentage points and a descriptive 95% t interval of 72.72–75.76%. All 20 pairs improved (range 68.77–79.79%); median completion times were 8,054.5 and 2,032.0 ms. Background CPU ticks fell by 97.69%, calculated from their two means. This is a measured service cost, not additional completed background work.

The archive also records 733 baseline and 14 ORCHESTRA package thermal-throttling counter increments. Consequently, the result describes the complete workload treatment, including changed background service and thermal behavior. It does not isolate a scheduler-only mechanism or demonstrate equal background throughput.

4.2 Fixed-work comparison

The fixed-work archive contains three repetitions per scheduler for each workload/worker cell. Table 2 gives arithmetic means and sample standard deviations. The ratio is the ORCHESTRA mean divided by the baseline mean; values above one indicate longer completion time [2].

Table 2: Fixed-work completion time, mean $\pm$ sample SD in milliseconds ($n = 3$ per scheduler per row). All ORCHESTRA rows recorded ownership.

Workload	Workers	Linux baseline	ORCHESTRA	Ratio
CPU	1	208.67 $\pm$ 1.53	330.00 $\pm$ 32.05	1.58
CPU	2	208.00 $\pm$ 1.73	418.67 $\pm$ 57.05	2.01
CPU	4	225.67 $\pm$ 30.60	461.67 $\pm$ 16.17	2.05
CPU	8	295.00 $\pm$ 75.03	511.00 $\pm$ 31.95	1.73
Mixed	1	1632.33 $\pm$ 286.88	1398.00 $\pm$ 336.63	0.86
Mixed	2	1958.67 $\pm$ 21.83	1843.00 $\pm$ 81.84	0.94
Mixed	4	1955.00 $\pm$ 27.84	1759.67 $\pm$ 74.78	0.90

Pure CPU work was slower with ORCHESTRA at every tested worker count. Mixed CPU/I/O work had lower mean completion times, with substantial variability in the one-worker cell. The small sample supports descriptive comparison; it does not establish broad statistical superiority.

4.3 Fresh CFS baseline and attachment gate

The 14 September campaign calibrated a fixed 600,000,000-iteration workload to approximately two seconds at one worker, then completed five independent CFS repetitions at each of one, two, and four workers (15/15 rows). The means $\pm$ sample SD were 2,039.00 $\pm$ 43.79, 1,885.40 $\pm$ 87.62, and 1,898.40 $\pm$ 30.24 ms, respectively; raw rows and command records are retained in the campaign archive [8]. The benchmark recorded an explicit ORCHESTRA block on every repetition before scheduler attachment: `sched_ext` was disabled, non-interactive `sudo -n true` required a password, and the exact matching kernel source needed for the BPF build was unavailable. No ORCHESTRA timing row was counted. These measurements therefore establish fresh CFS baseline reproducibility and an attachment-gate result, not a fresh ORCHESTRA performance comparison.

5 Runtime, stability, and control-path experiments

5.1 Attachment and action probes

Target-matched compilation, verifier acceptance, attachment, exact-task ownership, and clean unload completed on the recorded kernel. The capability report preserves 81/81 bounded runtime assertions. Representative observations are summarized in Table 3 [3].

Table 3: Observed action behavior in the archived runtime probes.

Action / probe	Recorded observation
RUN	31 dispatches, 34 running events, and 34 effective events; forward progress observed.
YIELD	45 specialized dispatches and 44 effective events; peer progress observed.
MIGRATE	28 accepted, 28 dispatched, and 27 effective events; requested, dispatched, and observed CPU were all 1.
Invalid target	44 bad-CPU fallbacks; zero MIGRATE acceptance or dispatch; effective RUN fallback.
SLEEP	Acceptance, deferral, release, and resumed progress.
THROTTLE	Immediate, after-RUN, rollover, republication, and two-CPU cases exercised.

5.2 Stability and recovery

CPU, I/O, and mixed stress phases completed at the five-second and thirty-second tiers. An extended ORCHESTRA run executed three consecutive 600-second phases: CPU, I/O, and mixed work, totaling 1,800 seconds. Each phase recorded task ownership and zero errors. The health scan detected no new panic, BUG, stall, hung-task, or RCU warning lines. Maximum sampled package temperature was 87 °C. The scheduler unloaded to the disabled state [4].

5.3 Signal publication and input handling

The documented userspace signal microbenchmark completed 12/12 valid runs: 12,000 successful publications and 27,161 verified reads, with zero reported HMAC, invalid-frame, or stale-frame failures in those runs. Separate userspace tamper tests rejected altered frames. Kernel-local probes rejected duplicate sequence values and exposed stale required-signal telemetry [2, 4].

The September recheck published and read back an externally supplied prediction-bearing frame. Readback recorded `prediction_used=1`, confidence 875, and predicted CPU 730. S1, S2, S3, S4, and Q fields were also populated. This experiment establishes record transport and readback; its values are not a prediction-accuracy measurement [6].

5.4 Real-time admission and regression checks

The archived FIFO, RR, and DEADLINE admission probes confirmed each Linux policy and recorded bridge rejection of adaptive opt-in and publication (return code 6), followed by clean unload. Separate FIFO/RR contention probes recorded progress of an ORCHESTRA-owned normal task. These observations concern the admission boundary and tested coexistence arrangement [4].

The 3 September archive preserves a tick-10 rejected-frame integration failure [5]. The 14 September campaign independently passed 30/30 named unit tests, the integration suite (including 34 CSV validator tests), and security regressions. A recorded publication stress pass produced 2,000 generations and 7,997 verified reads without invalid-frame reads or regressions. The v7 userspace pipeline completed 6/6 invocations, each with 40 rows. These are userspace validation results. The aggregate `make check/make test` gate still stopped at an absolute documentation path flagged by the repository security scan [8].

6 Discussion

6.1 Engineering contribution relative to the architecture

The implemented interfaces make the architecture experimentally observable: task admission identifies the subject, publication identifies the directive and generation, and telemetry records the execution outcome. Table 4 relates the delivered mechanisms to the source paper. The distinction between userspace evidence and kernel execution is essential when assessing implementation progress.

Table 4: Architecture-to-implementation mapping for tested mechanisms.

Architectural element	Software realization	Observed evidence
Adaptive actions	Bridge directives and compact BPF executor	Ownership, execution, CPU placement, deferral and release probes.
Signal dissemination	Userspace publication plus versioned runtime records	Publication/read validation; prediction-bearing record readback.
Integrity and freshness	Userspace authenticated frames; kernel-local sequence checks	Tamper rejection and duplicate/stale handling within their respective trust boundaries.
Safety boundary	RT opt-in rejection and fallback handling	FIFO/RR/DEADLINE admission probes; bounded coexistence and clean unload.

6.2 Interpretation of the measured trade-off

Foreground protection and fixed-work throughput answer different questions. The paired experiment shows that explicit background service control can substantially improve a selected foreground task’s completion time. Table 2 shows that the evaluated scheduler incurs a substantial pure-CPU completion-time cost. Reporting both identifies a concrete use case and its operating trade-off.

The workflow and action probes establish an observable control path: identity admission, versioned publication, task execution, telemetry, and unload. The safety and signal experiments add evidence that particular invalid inputs are rejected and that the tested lifecycle returns control to Linux. These are specific behavioral findings tied to the recorded artifacts.

6.3 Interpretive limits of the experiments

The measurements come from one host and recorded workload protocols. The fixed-work sample has three repetitions per cell, and the September host had background activity. In the paired experiment, service budgets and thermal behavior changed together. A comparison with an equally constrained Linux cgroup policy was not part of that protocol, so the measurements do not attribute all benefit uniquely to ORCHESTRA. The 30-minute stability result establishes behavior for its three observed phases; it is not a long-term reliability estimate. Userspace authentication tests and local kernel freshness checks exercise different trust boundaries.

7 Conclusion

The existing experiments demonstrate a functioning opt-in Linux scheduler with task-ownership evidence, five observable scheduling actions, and clean lifecycle recovery. Foreground completion improved by 74.24% on average across 20 pairs while background CPU service decreased by 97.69%. Pure-CPU fixed-work performance was slower than the Linux baseline; the small mixed-workload sample had lower mean completion times. Stability, signal, admission, and regression experiments complement these measurements with operational evidence. Together, the results support ORCHESTRA-OS as a scheduling research platform with a demonstrated foreground-service policy and measurable performance costs.

8 Evidence traceability and research attribution

8.1 Relationship to the source architecture

The source paper specifies prediction, integrity-protected signals, per-process adaptation, coordination measurement, and hierarchical control. Its revised coordination expression is $Q = (S_1 S_2 S_3 S_4)^{1/4}$, with fidelity/freshness, directive compliance, action coherence, and temporal stability as its factors [1]. In this report, populated fields establish transport evidence only. Kernel action probes support `KERNEL_PROTOTYPED` claims; repeated foreground trials support `EXPERIMENTALLY_VALIDATED` behavior within their protocol. Independent software tests support `USERSPACE_VALIDATED` claims. The overall implementation is `PARTIALLY_VALIDATED`.

8.2 Evidence register

Paths below are relative to the companion evidence repository, <https://github.com/Aether-0/ORCHESTRA-OS-evidence>. The identifiers E1–E6 are used throughout this table so that each conclusion has one unambiguous evidence pointer. Each pointer identifies an existing source, with raw records retained for examination. Checksums establish file integrity; they do not independently establish measurement validity. The repository is private during review; access may be granted to the editor, reviewers, or assessors.

Table 5: Traceable evidence for the principal conclusions.

Claim	Evidence location and scope
20 paired trials	[E1] <code>E1_foreground_pairs/photo_archival_pairs.csv</code> ; all 40 paired-run rows, ownership and thermal counters.
Fixed-work comparison	[E2] <code>E2_fixed_work/fixed_work_raw/</code> and <code>E2_fixed_work/fixed_work_summary.csv</code> ; three runs per scheduler and worker cell.
Actions and lifecycle	[E3] <code>E3_actions_lifecycle/</code> ; capability report and supporting runtime logs; 81 bounded assertions.
Fresh validation	[E4] <code>E4_fresh_validation/</code> ; unit, integration, security, aggregate-gate and build logs from the 14 September campaign.
Fresh CFS baseline	[E5] <code>E5_fresh_cfs_baseline/</code> ; five repetition directories per worker count, raw CSVs and command logs; no ORCHESTRA timing rows were produced.
Architecture basis	[E6] <code>E6_architecture_basis/PROFESSOR_ARCHITECTURE_PAPER_20260706.pdf</code> ; exact copy of the supplied 6 July simulation-study submission draft.

8.3 Reproducibility and availability

All results in this paper are drawn from the existing project reports and archived measurements. The companion evidence repository retains the source documents, command records, and data files in the E1–E6 directories. The principal numerical sources are `E2_fixed_work/fixed_work_summary.csv` and `E1_foreground_pairs/photo_archival_statistics.txt`; the fresh CFS baseline is recorded under `E5_fresh_cfs_baseline/`. This paper adds no new runtime measurements.

References

- [1] G. Ganesan and P. Vaithyanathan, *ORCHESTRA-OS: Simulation-Driven Architectural Discovery and Design Principles for Predictive, Hierarchical, Signal-Coordinated Process Scheduling*, submission draft prepared for IEEE Transactions on Parallel and Distributed Systems, 6 July 2026. Supplied manuscript; publication acceptance is not asserted.
- [2] S.W. ZAW, *ORCHESTRA-OS versus Linux CFS: Evidence-Based Strengths, Weaknesses, and Field Fit*, validation report, 1 September 2026.
- [3] S.W. ZAW, *ORCHESTRA-OS Final Product and Capability Assessment Report*, 2 September 2026.
- [4] ORCHESTRA-OS archive, *Real-World Validation and Fix Report*, 28 August 2026.
- [5] ORCHESTRA-OS archive, *Real-World Checklist Execution Report*, 3 September 2026.
- [6] ORCHESTRA-OS archive, *No-Block Checklist Report*, 3 September 2026.
- [7] ORCHESTRA-OS, *sched_ext Runtime Documentation*, repository file `kernel/sched_ext/README.md`.
- [8] ORCHESTRA-OS, *Fresh CFS baseline and userspace validation*, evidence campaign, 14 September 2026.